AI-powered Virtual Humans

LLM-Powered Cognitive Architectures

Module 2 - Chapter 02
Memory and the Virtual Human



From a List to a Checkpointer [1] [3]

Module 1: your list

Append human and AI messages. Resend the whole list on each call.

LangGraph

Messages live in the graph state. A checkpointer saves them per conversation.

Same idea; the graph manages it for us.

Messages in State [2]

```
from typing import Annotated
from langgraph.graph.message import add_messages

class ChatState(TypedDict):
    messages: Annotated[list, add_messages]
```

Annotated attaches a reducer to the messages key.

Replace or Append [2]

Default (Chapter 01)

A new value replaces the old one.

add_messages

New messages are appended to the list.

MessagesState provides the same messages key and reducer, ready-made.

Open 01_memory.ipynb: 1. Messages in state

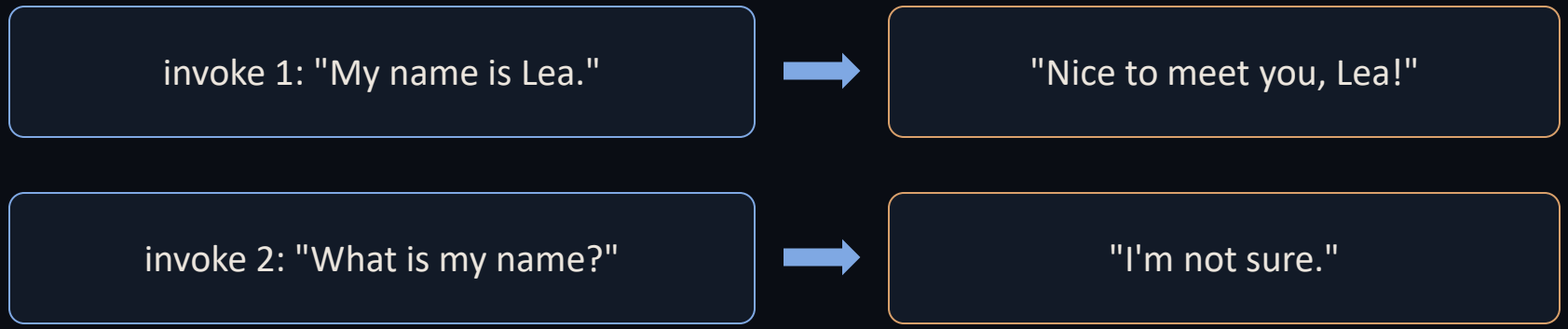
A Chatbot Node [2]

```
prompt = ChatPromptTemplate.from_messages([
    SystemMessagePromptTemplate.from_template(
        "You are a museum guide. Reply briefly."),
    MessagesPlaceholder("messages"),
])

def chatbot(state: MessagesState):
    messages = prompt.format_messages(messages=state["messages"])
    return {"messages": [llm.invoke(messages)]}
```

The node sends the instruction and the messages; it returns only the reply.

Without a Checkpointer



Earlier calls are not restored.

Open 01_memory.ipynb: 2. A chatbot node

Checkpointer and Threads [1]

Checkpoint

A snapshot of the graph state, saved after each step.

Thread

One conversation, identified by a thread_id.

Same thread_id: the saved state is restored before the graph runs.

Compile with a Checkpointer [1]

```
from langgraph.checkpoint.memory import InMemorySaver

saver = InMemorySaver()
memory_graph = builder.compile(checkpointer=saver)
lea = {"configurable": {"thread_id": "lea"}}

memory_graph.invoke({"messages": [HumanMessage("My name is Lea.")]}, lea)
memory_graph.invoke({"messages": [HumanMessage("What is my name?")]}, lea)
```

Send only the new message; the earlier ones are restored.

Separate Conversations [1]

thread_id = "lea"

My name is Lea.
What is my name? → Lea

thread_id = "marco"

What is my name? → I'm not sure.

A new thread_id starts an empty conversation.

Open 01_memory.ipynb: 3. Checkpointer and threads

Where Checkpoints Live [1]

InMemorySaver

Kept in the Python process.
Lost when the kernel or server restarts.

Persistent savers

SQLite or Postgres.
Conversations survive restarts.

Same graph code; only the checkpointer changes.

Delete a Saved Conversation [1]

```
saver.delete_thread("lea")
print(memory_graph.get_state(lea).values)

# The next call starts a new conversation.
result = memory_graph.invoke(
    {"messages": [HumanMessage("What is my name?")]}, lea)
```

Delete one thread. Other conversations keep their state.

A Rolling Buffer [3]

```
MAX_MESSAGES = 6

def buffered_chatbot(state: MessagesState):
    recent = state["messages"][-MAX_MESSAGES:]
    reply = llm.invoke(prompt.format_messages(messages=recent))
    return {"messages": [reply]}
```

The state keeps everything; the model sees only the latest messages.

Exercise: Two Visitors

Two threads

Lea likes green, Marco likes blue.
Alternate between their threads.
Ask each colour back.

A short buffer

Set MAX_MESSAGES = 2.
Inspect the recent messages.
Compare with the saved state.

Open 01_memory.ipynb: 4. A rolling buffer + exercise

The Same Graph in the Avatar

Notebook

Messages in state.
A checkpointer saves each thread.
The model returns text.

Avatar agent

Keep the same memory.
Return ModelReply for speech, emotion, expression and gesture.

Open agents/Module_2/Chapter_02/langgraph_avatar_agent.py

State: Messages and Reply [2]

```
class AvatarState(MessagesState):
    reply: dict
```

messages

The conversation, appended by the reducer.
Only the spoken text is stored.

reply

The full ModelReply:
speech and avatar behaviour.

The respond Node [4]

```
def respond(state: AvatarState):
    current_time = datetime.now().strftime("%H:%M")
    answer = structured_client.invoke(
        prompt.format_messages(
            current_time=current_time,
            messages=state["messages"][-MAX_MESSAGES:]))
    return {"messages": [AIMessage(answer.speech_text)],
            "reply": answer.model_dump()}
```

Structured output from Module 1 + the rolling buffer we just built.

The Thread Lives in the Backend [1]

```
THREAD_ID = "avatar"
config = {"configurable": {"thread_id": THREAD_ID}}

def response(text: str) -> AgentReply:
    result = graph.invoke({"messages": [HumanMessage(text)]}, config)
    return AgentReply(**result["reply"])
```

The browser sends only text; every request continues the same thread.

Clear Conversation [1]




```
def clear_conversation():
    checkpointer.delete_thread(THREAD_ID)
```

The next message starts an empty conversation.

Together: Clear Memory through the API

```
@app.post("/conversation/clear")
def clear_conversation():
    selected_agent = sys.modules[response.__module__]
    clear_agent_memory = getattr(
        selected_agent, "clear_conversation", None)
    if callable(clear_agent_memory):
        clear_agent_memory()
    return {"status": "ok"}
```

Rebuild this route together, then test Clear conversation.

Select and Test the Avatar Agent

In vh_cognitive_server.py keep this response import active:

```
from agents.Module_2.Chapter_02.langgraph_avatar_agent import response
```

Tell your name and favourite colour, then ask for both.
Request a happy greeting with a raised hand.
Clear the conversation and ask for your name again.

Save, then restart the cognitive server in the avatar Settings.

Your Turn

Persona

Change the system instruction: a museum guide with its own name.

Buffer

Set MAX_MESSAGES = 4. Which messages reach the model?

Optional

Return a different gesture when greeting and when saying goodbye.

Edit the agent file, save and restart the cognitive server each time.

Next: Tools and Coding Agents

Our avatar remembers, speaks and animates.

Chapter 03 adds current weather through a tool.
We use a coding agent to build and extend it.

References

[1] LangGraph persistence

Checkpoints, threads and saved state

[2] LangGraph Graph API

Reducers and MessagesState

[3] LangGraph memory

Short-term memory and trimming messages

[4] ChatOllama structured output

Structured speech and avatar behaviour

[5] Chat prompt templates

System instructions and message history